

Módulo Kids — Gestão de entrega de crianças nas salinhas

Status: proposta técnica (nada implementado ainda) · **Data:** 2026-08-25 · **Branch alvo:** a definir

Módulo para controlar a entrega (check-in), o acompanhamento e a devolução (check-out) de crianças nas salas de aula durante o culto, com registro de anotações e um canal de alerta escalonado para os responsáveis.

Princípio herdado do SaaS: o módulo é **aditivo**. Nenhum fluxo atual (células, visitantes, presença, materiais) pode mudar de comportamento por causa do Kids. Todo dado novo é isolado por `churchId`, como o restante.

Sumário

1. Escopo e glossário
 2. Restrições técnicas que mudam o desenho
 3. Papéis e permissões
 4. Modelo de dados (Prisma)
 5. Máquinas de estado
 6. Fluxos
 7. API REST
 8. Alertas e notificações
 9. Segurança
 10. LGPD e dados de saúde
 11. App Flutter
 12. Plano SaaS e feature gating
 13. Impacto no código existente
 14. Fases de entrega
 15. Testes e critérios de aceite
 16. Observabilidade e operação
 17. Riscos e decisões pendentes
-

1. Escopo e glossário

1.1 O que o módulo faz

Capacidade	Descrição
Cadastro de salas	Nome, capacidade, faixa etária, professores vinculados
Cadastro de crianças	Via app do responsável ou cadastro rápido feito pelo professor
Check-in	Leitura do QR Code do app do responsável ou cadastro rápido + senha de 5 dígitos

Capacidade	Descrição
Ocupação em tempo real	Quantas crianças há na sala vs. capacidade
Anotações	Individuais (por criança) e gerais (por aula)
Alertas	3 níveis, com push, WhatsApp, ligação e alerta crítico
Check-out	QR Code do responsável ou senha de 5 dígitos
Histórico	Quem entregou, quem retirou, quando, por qual método

1.2 O que **não** faz (fora de escopo v1)

- Controle financeiro/mensalidade de escola infantil.
- Currículo/plano de aula (materiais já existem no módulo de células).
- Reconhecimento facial ou biometria.
- Impressão de etiquetas térmicas (avaliado na §17 como item futuro).

1.3 Glossário

Termo	Significado
Sala (<i>KidsRoom</i>)	Espaço físico com capacidade e professores. Ex.: "Berçário", "Kids 4-6"
Sessão (<i>KidsSession</i>)	Uma aula: sala + data/culto. É o que abre e fecha
Responsável (<i>KidsGuardian</i>)	Quem entrega/retira. Pode ou não ter conta no app
Check-in (<i>KidsCheckin</i>)	Registro de uma criança dentro de uma sessão
Senha de retirada	Código de 5 dígitos gerado no check-in de quem não tem app
Alerta (<i>KidsAlert</i>)	Comunicação disparada da sala para o responsável

2. Restrições técnicas que mudam o desenho

Três pontos precisam ser decididos antes do código, porque o pedido original assume infraestrutura que **hoje não existe** no projeto.

2.1 Push notification não está implementado

`firebase_messaging` e `firebase_core` estão no `pubspec.yaml`, mas **nenhuma linha de Dart usa `FirebaseMessaging`**, e a API **não tem `firebase-admin`**. As "notificações" atuais (`model Notification`) são in-app: gravadas no banco e lidas na tela `/notifications`.

Consequência: "o pai recebe uma notificação" exige construir a stack de push do zero — é a Fase 0 (§14). Sem ela, o Nível 1 de alerta vira apenas item na tela de notificações, o que não serve para uma sala de aula.

2.2 WhatsApp hoje é link, não API

O envio atual (`admin_whatsapp_page.dart:922`) monta `https://wa.me/<fone>?text=...` e abre o WhatsApp **do próprio usuário** via `url_launcher`. É um envio manual, com uma pessoa apertando "enviar" — inaceitável para alerta automático.

Consequência: alerta por WhatsApp exige **WhatsApp Cloud API** (Meta) ou um BSP, com **template de utilidade pré-aprovado**, porque a janela de 24 h de conversa não estará aberta. Ver §8.3.

2.3 "Alerta de sinistro tipo Defesa Civil" não é possível para um app

O alerta da Defesa Civil usa **Cell Broadcast / WEA**, um canal da operadora reservado por lei a autoridades públicas. **Nenhum app de terceiro consegue emitir esse alerta** — nem com permissão do usuário, nem pagando à operadora.

O que dá para fazer, e que é o mais próximo disso em termos de "furar" o silencioso do celular:

Plataforma	Recurso	Requisito
Android	Canal <code>IMPORTANCE_HIGH</code> + <code>fullScreenIntent</code> + <code>CATEGORY_CALL</code> + som contínuo; tela cheia mesmo bloqueado	Permissão <code>USE_FULL_SCREEN_INTENT</code> (Android 14+ pede aprovação na Play Store para apps não-telefonia)
Android	Bypass do Não Perturbe	Usuário concede <code>ACCESS_NOTIFICATION_POLICY</code> uma vez, no onboarding
iOS	Critical Alerts — toca mesmo no silencioso/foco	Entitlement especial aprovado pela Apple caso a caso; aprovação não é garantida
iOS (fallback)	<code>interruptionLevel: .timeSensitive</code>	Sem entitlement especial; fura o Foco, não o silencioso
Ambos	Push VoIP + CallKit/ConnectionService: o alerta toca como uma ligação	Certificado VoIP (iOS); uso fora de VoIP real pode ser rejeitado pela Apple
Ambos	Ligação telefônica de verdade (<code>tel:</code> no app do professor)	Nenhum — já temos <code>url_launcher</code>

Recomendação: Nível 3 = **ligação telefônica imediata** (ação humana, confiável) + push crítico (full-screen no Android, time-sensitive no iOS) + WhatsApp, os três em paralelo. Pedir o entitlement de Critical Alerts à Apple em paralelo ao desenvolvimento, tratando a aprovação como incerta. Documentar isso ao cliente para não vender o que a plataforma não entrega.

3. Papéis e permissões

3.1 Papéis novos

Dois papéis entram no `enum UserRole` (Prisma + `user_entity.dart`):

- **KIDS** — professor/tio da salinha. Opera check-in, check-out, anotações e alertas **apenas das salas em que está vinculado**.
- **RESPONSAVEL** — pai/mãe/responsável. Conta de escopo mínimo: vê os próprios filhos, gera o QR Code, recebe alertas, lê anotações liberadas.

ADMIN administra salas e vê tudo do módulo; **KIDS_COORD** (opcional, §17) seria o supervisor do ministério infantil.

3.2 Matriz

Ação	RESPONSAVEL	KIDS	ADMIN	SUPERADMIN
Cadastrar/editar sala	—	—	✓	✓
Vincular professor à sala	—	—	✓	✓
Abrir/fechar sessão	—	✓ (sua sala)	✓	✓
Check-in / check-out	—	✓ (sua sala)	✓	✓
Cadastro rápido de criança	—	✓	✓	✓
Anotação individual	ler ¹	✓	✓	✓
Anotação geral da aula	ler	✓	✓	✓
Alerta nível 1 e 2	receber	✓	✓	✓
Alerta nível 3 (emergência)	receber	✓	✓	✓
Ver dados de saúde da criança	✓ (filho)	✓ (em sala)	✓	✓
Gerar QR do responsável	✓	—	—	—
Relatórios do ministério	—	—	✓	✓

¹ Anotação individual tem campo **visibleToGuardian** — o professor decide se a observação vai para o responsável ou fica como registro interno.

Regra de ouro do KIDS: todo endpoint valida que o professor está em **KidsRoomTeacher** da sala da sessão. Não basta o papel.

4. Modelo de dados (Prisma)

4.1 Enums

```
enum KidsSessionStatus { OPEN CLOSED }
enum KidsCheckinStatus { CHECKED_IN CHECKED_OUT NO_SHOW }
enum KidsCheckMethod   { QR CODE MANUAL }           // MANUAL = liberação
com justificativa
enum KidsNoteKind       { INDIVIDUAL CLASS }
enum KidsAlertLevel     { INFO URGENT EMERGENCY }    // níveis 1, 2 e 3
enum KidsAlertStatus    { OPEN ACKNOWLEDGED RESOLVED }
```

```
enum KidsChannel      { PUSH WHATSAPP SMS CALL CRITICAL_PUSH }
enum KidsDeliveryStatus { QUEUED SENT DELIVERED READ FAILED }
enum KidsTeacherRole  { TITULAR AUXILIAR }
enum GuardianRelation { PAI MAE AVO TIO RESPONSAVEL_LEGAL OUTRO }
```

4.2 Sala e professores

```
model KidsRoom {
  id          String    @id @default(uuid())
  churchId    String?   @map("church_id")
  name        String
  description String?
  capacity    Int
  minAgeMonths Int?     @map("min_age_months") // faixa etária em meses:
  berçário precisa
  maxAgeMonths Int?     @map("max_age_months")
  color       String    @default("#3F51B5")
  isActive    Boolean  @default(true) @map("is_active")
  createdAt   DateTime @default(now()) @map("created_at")
  updatedAt   DateTime @updatedAt @map("updated_at")

  church Church? @relation(fields: [churchId], references:
[id])
  teachers KidsRoomTeacher[]
  sessions KidsSession[]

  @@index([churchId])
  @@map("kids_rooms")
}

model KidsRoomTeacher {
  id          String    @id @default(uuid())
  churchId    String?   @map("church_id")
  roomId      String    @map("room_id")
  userId      String    @map("user_id")
  role        KidsTeacherRole @default(AUXILIAR)
  createdAt   DateTime @default(now()) @map("created_at")

  church Church? @relation(fields: [churchId], references: [id])
  room    KidsRoom @relation(fields: [roomId], references: [id], onDelete:
Cascade)
  user    User      @relation("KidsTeacher", fields: [userId], references:
[id], onDelete: Cascade)

  @@unique([roomId, userId])
  @@index([churchId])
  @@index([userId])
  @@map("kids_room_teachers")
}
```

4.3 Sessão (a aula)

```
model KidsSession {
  id          String          @id @default(uuid())
  churchId    String?         @map("church_id")
  roomId      String          @map("room_id")
  serviceDate DateTime        @map("service_date") @db.Date
  serviceName String?         @map("service_name") // "Culto da manhã"
  status      KidsSessionStatus @default(OPEN)
  openedById  String          @map("opened_by_id")
  openedAt    DateTime        @default(now()) @map("opened_at")
  closedById  String?         @map("closed_by_id")
  closedAt    DateTime?       @map("closed_at")
  lesson      String?         // tema da aula
  capacityOverride Int?       @map("capacity_override")

  church  Church?    @relation(fields: [churchId], references: [id])
  room    KidsRoom   @relation(fields: [roomId], references: [id])
  openedBy User       @relation("KidsSessionOpened", fields:
[openedById], references: [id])
  checkins KidsCheckin[]
  notes    KidsNote[]
  alerts   KidsAlert[]

  @@unique([roomId, serviceDate, serviceName])
  @@index([churchId])
  @@index([roomId, serviceDate])
  @@map("kids_sessions")
}
```

`@@unique([roomId, serviceDate, serviceName])` impede duas sessões concorrentes da mesma sala no mesmo culto — erro comum quando dois professores abrem a sala ao mesmo tempo.

4.4 Criança e responsáveis

O `model Child` **já existe** (filhos cadastrados no perfil do usuário: `userId`, `name`, `birthDate`). O módulo o estende — `userId` passa a ser **opcional**, porque criança cadastrada no balcão não tem responsável com conta.

```
model Child {
  id          String          @id @default(uuid())
  churchId    String?         @map("church_id")
  userId      String?         @map("user_id") // ← passa a ser
nullable
  name        String
  birthDate   DateTime? @map("birth_date")
  gender      Gender?
  photoKey    String?        @map("photo_key") // MinIO; opcional, ver
§10
```

```

    allergies      String? @db.Text          // dado sensível
    medications    String? @db.Text          // dado sensível
    disabilities   String? @db.Text          // dado sensível
    medicalNotes   String? @db.Text          // dado sensível
    authorizedPickup String? @db.Text        // quem mais pode
retirar
    isActive       Boolean @default(true) @map("is_active")
    createdAt      DateTime @default(now()) @map("created_at")
    updatedAt      DateTime @updatedAt @map("updated_at")

    church         Church? @relation(fields: [churchId], references: [id])
    user           User?   @relation(fields: [userId], references: [id],
onDelete: SetNull)
    guardians      KidsGuardian[]
    checkins       KidsCheckin[]
    notes          KidsNote[]
    alerts         KidsAlert[]

    @@index([churchId])
    @@index([userId])
    @@map("children")
}

model KidsGuardian {
  id          String          @id @default(uuid())
  churchId    String?         @map("church_id")
  childId     String          @map("child_id")
  userId      String?         @map("user_id")      // preenchido se tiver
conta no app
  name        String
  phone       String          // E.164:
+5511999999999
  hasWhatsapp Boolean         @default(true) @map("has_whatsapp")
  relation     GuardianRelation @default(RESPONSABLE_LEGAL)
  isPrimary    Boolean         @default(false) @map("is_primary")
  canPickup    Boolean         @default(true) @map("can_pickup")
  createdAt    DateTime        @default(now()) @map("created_at")

  church Church? @relation(fields: [churchId], references: [id])
  child  Child   @relation(fields: [childId], references: [id], onDelete:
Cascade)
  user   User?   @relation("KidsGuardian", fields: [userId], references:
[id], onDelete: SetNull)

  @@index([churchId])
  @@index([childId])
  @@index([userId])
  @@map("kids_guardians")
}

```

4.5 Check-in / check-out

```

model KidsCheckin {
  id          String          @id @default(uuid())
  churchId    String?         @map("church_id")
  sessionId   String          @map("session_id")
  childId     String          @map("child_id")
  status      KidsCheckinStatus @default(CHECKED_IN)
  badgeCode   String          @map("badge_code")    // sequencial legível
da sessão: "K-014"

  checkinAt    DateTime        @default(now()) @map("checkin_at")
  checkinById  String          @map("checkin_by_id")
  checkinMethod KidsCheckMethod @map("checkin_method")
  checkinGuardianId String?     @map("checkin_guardian_id")

  // Senha de retirada – apenas para quem não usa QR. Nunca em texto puro.
  pickupCodeHash String? @map("pickup_code_hash")
  pickupCodeLast2 String? @map("pickup_code_last2") // conferência
visual no balcão
  pickupAttempts Int @default(0) @map("pickup_attempts")
  pickupLockedUntil DateTime? @map("pickup_locked_until")

  checkoutAt    DateTime? @map("checkout_at")
  checkoutById  String? @map("checkout_by_id")
  checkoutMethod KidsCheckMethod? @map("checkout_method")
  checkoutGuardianId String? @map("checkout_guardian_id")
  checkoutGuardianName String? @map("checkout_guardian_name") //
liberação MANUAL
  checkoutReason String? @map("checkout_reason") //
justificativa da MANUAL

  church Church? @relation(fields: [churchId], references: [id])
  session KidsSession @relation(fields: [sessionId], references: [id],
onDelete: Cascade)
  child Child @relation(fields: [childId], references: [id])
  notes KidsNote[]
  alerts KidsAlert[]

  @@unique([sessionId, childId]) // a mesma criança não entra duas
vezes
  @@unique([sessionId, badgeCode])
  @@index([churchId])
  @@index([status])
  @@map("kids_checkins")
}

```

4.6 Anotações

```

model KidsNote {
  id          String          @id @default(uuid())
  churchId    String?         @map("church_id")

```

```

    sessionId String @map("session_id")
    kind KidsNoteKind
    childId String? @map("child_id") // null quando kind = CLASS
    checkinId String? @map("checkin_id")
    body String @db.Text
    visibleToGuardian Boolean @default(true) @map("visible_to_guardian")
    authorId String @map("author_id")
    createdAt DateTime @default(now()) @map("created_at")
    updatedAt DateTime @updatedAt @map("updated_at")

    church Church? @relation(fields: [churchId], references: [id])
    session KidsSession @relation(fields: [sessionId], references: [id],
onDelete: Cascade)
    child Child? @relation(fields: [childId], references: [id])
    checkin KidsCheckin? @relation(fields: [checkinId], references: [id])
    author User @relation("KidsNoteAuthor", fields: [authorId],
references: [id])

    @@index([churchId])
    @@index([sessionId, kind])
    @@index([childId])
    @@map("kids_notes")
}

```

4.7 Alertas e entregas

```

model KidsAlert {
  id String @id @default(uuid())
  churchId String? @map("church_id")
  sessionId String @map("session_id")
  childId String @map("child_id")
  checkinId String? @map("checkin_id")
  level KidsAlertLevel
  status KidsAlertStatus @default(OPEN)
  message String @db.Text
  createdById String @map("created_by_id")
  createdAt DateTime @default(now()) @map("created_at")
  acknowledgedAt DateTime? @map("acknowledged_at")
  acknowledgedByGuardianId String? @map("acknowledged_by_guardian_id")
  resolvedAt DateTime? @map("resolved_at")
  resolvedById String? @map("resolved_by_id")

  church Church? @relation(fields: [churchId], references:
[id])
  session KidsSession @relation(fields: [sessionId], references:
[id], onDelete: Cascade)
  child Child @relation(fields: [childId], references:
[id])
  checkin KidsCheckin? @relation(fields: [checkinId], references:
[id])
  deliveries KidsAlertDelivery[]
}

```

```

    @@index([churchId])
    @@index([sessionId, status])
    @@map("kids_alerts")
}

model KidsAlertDelivery {
  id          String          @id @default(uuid())
  churchId    String?         @map("church_id")
  alertId     String          @map("alert_id")
  guardianId  String?         @map("guardian_id")
  userId      String?         @map("user_id")
  channel     KidsChannel
  status      KidsDeliveryStatus @default(QUEUED)
  providerMessageId String?    @map("provider_message_id")
  error       String?         @db.Text
  attempts    Int             @default(0)
  queuedAt    DateTime        @default(now()) @map("queued_at")
  sentAt      DateTime?       @map("sent_at")
  deliveredAt DateTime?       @map("delivered_at")
  readAt      DateTime?       @map("read_at")

  church  Church?    @relation(fields: [churchId], references: [id])
  alert   KidsAlert  @relation(fields: [alertId], references: [id],
onDelete: Cascade)
  guardian KidsGuardian? @relation(fields: [guardianId], references: [id])

  @@index([churchId])
  @@index([alertId])
  @@index([status])
  @@map("kids_alert_deliveries")
}

```

4.8 Infra de push e antifraude do QR

```

model DeviceToken {
  id          String  @id @default(uuid())
  churchId    String? @map("church_id")
  userId      String  @map("user_id")
  token       String  @unique
  platform    String                                     // android | ios | web
  appVersion  String? @map("app_version")
  criticalOptIn Boolean @default(false) @map("critical_opt_in") //
  permissão de alerta crítico
  lastSeenAt  DateTime @default(now()) @map("last_seen_at")
  createdAt   DateTime @default(now()) @map("created_at")

  church Church? @relation(fields: [churchId], references: [id])
  user   User    @relation("UserDevices", fields: [userId], references:
[id], onDelete: Cascade)
}

```

```

    @@index([churchId])
    @@index([userId])
    @@map("device_tokens")
  }

  /// Nonce do QR do responsável – impede que um print do QR seja reusado.
  model KidsQrNonce {
    jti      String   @id
    userId   String   @map("user_id")
    usedAt   DateTime @default(now()) @map("used_at")
    expiresAt DateTime @map("expires_at")

    @@index([expiresAt])
    @@map("kids_qr_nonces")
  }

```

4.9 Migração

```

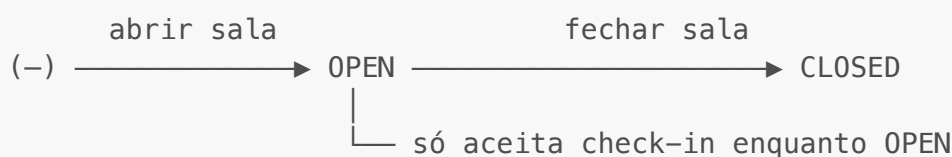
0003_kids_module
- CreateEnum (9 enums novos)
- CreateTable (kids_rooms, kids_room_teachers, kids_sessions,
kids_guardians,
               kids_checkins, kids_notes, kids_alerts,
kids_alert_deliveries,
               device_tokens, kids_qr_nonces)
- AlterTable children:
  ALTER COLUMN user_id DROP NOT NULL
  ADD COLUMN gender, photo_key, allergies, medications, disabilities,
  medical_notes, authorized_pickup, is_active, updated_at
- AlterEnum   UserRole: ADD VALUE 'KIDS', ADD VALUE 'RESPONSAVEL'
- Backfill    children.church_id já existe; nada a preencher

```

ALTER TYPE ... ADD VALUE não roda dentro de transação no Postgres < 12 e exige commit separado no 12+. A migração dos enums vai em arquivo próprio, antes do resto.

5. Máquinas de estado

5.1 Sessão



Fechar sessão com criança **CHECKED_IN** é **bloqueado** (409). O professor precisa fazer check-out de todas ou registrar liberação **MANUAL** com justificativa.

5.2 Check-in

CHECKED_IN — check-out válido —> CHECKED_OUT

└ sessão fechada sem retirada —> (bloqueado; exige MANUAL + motivo)

NO_SHOW: criança pré-agendada que não apareceu (uso futuro, §17)

5.3 Alerta

OPEN — responsável abre/confirma —> ACKNOWLEDGED — professor encerra —> RESOLVED

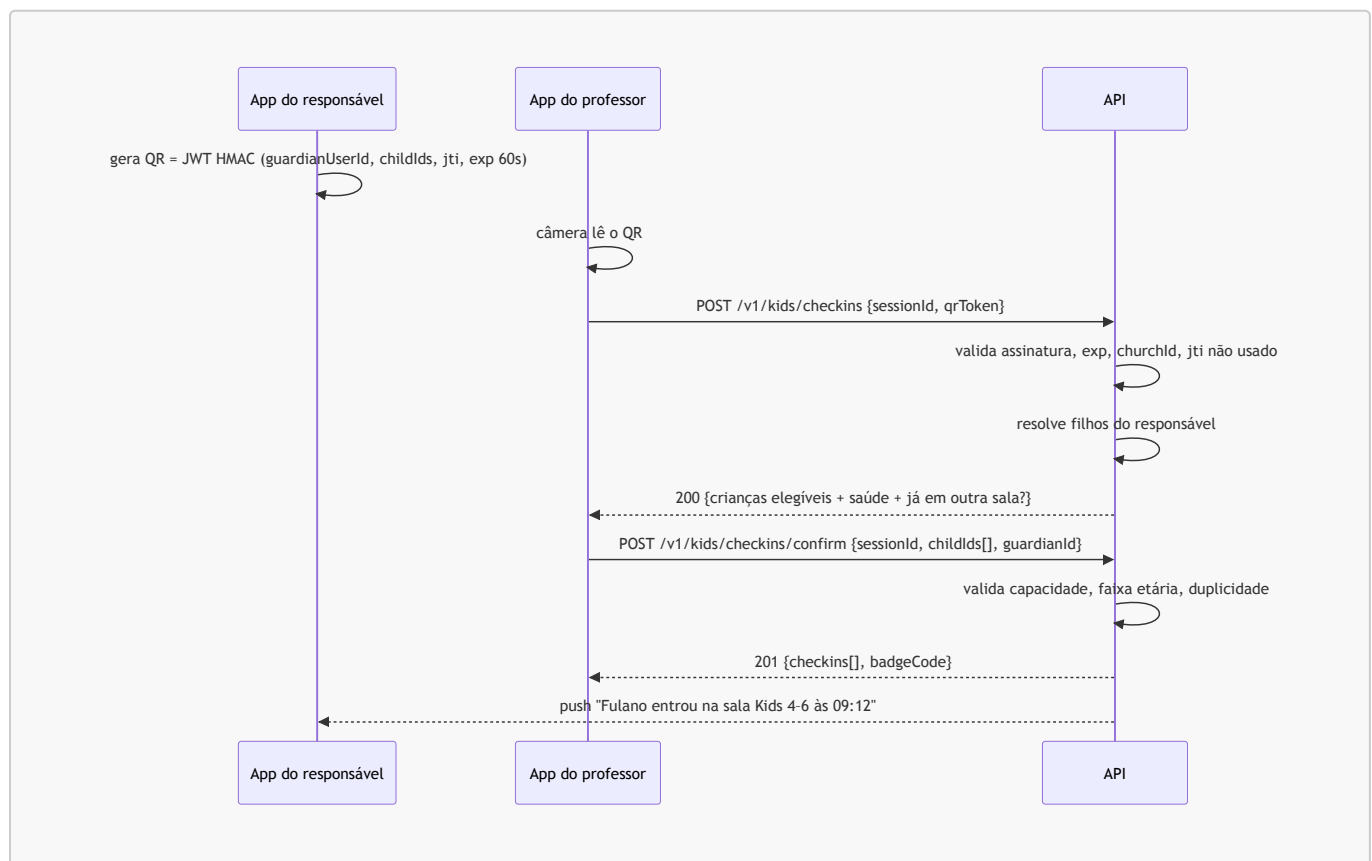
▲

└ professor encerra sem confirmação do pai

Alerta **EMERGENCY** **não** é auto-resolvido: exige ação explícita de quem abriu.

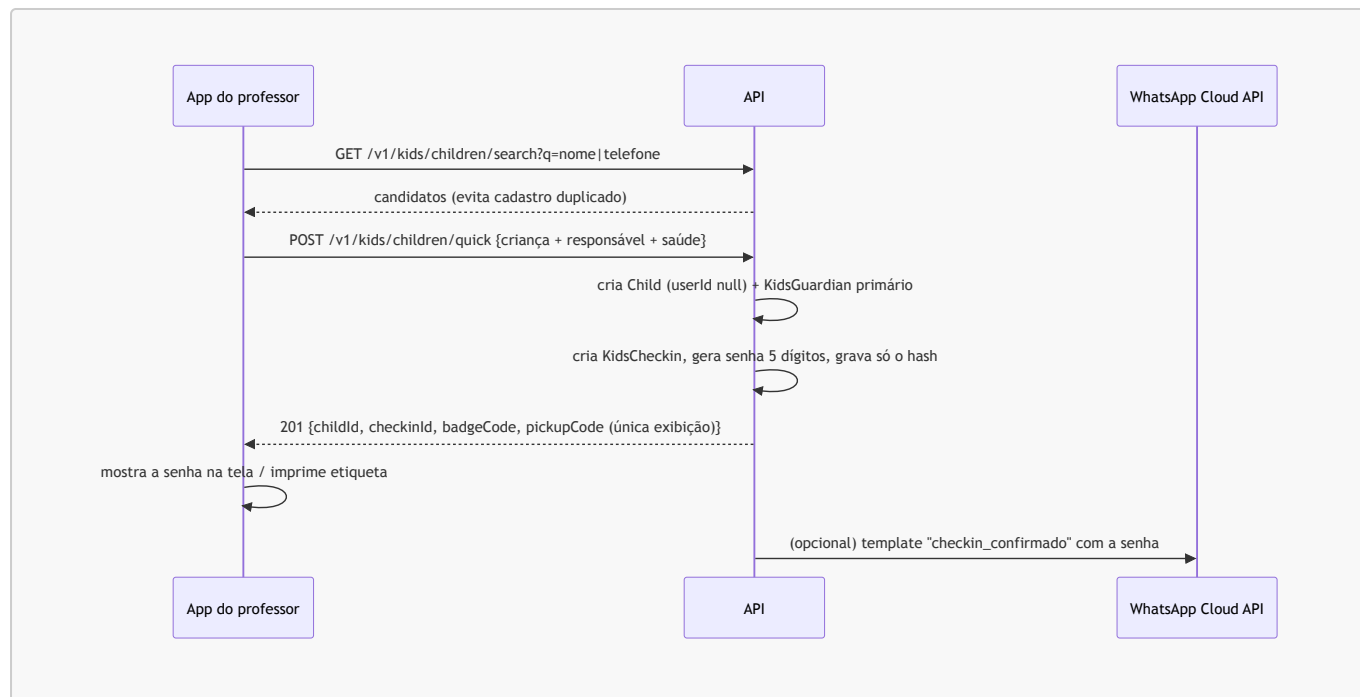
6. Fluxos

6.1 Check-in com QR (responsável tem o app)



Se o responsável tem app, **não há senha de 5 dígitos**: a retirada usa o mesmo QR. Isso é o caminho seguro e deve ser o incentivado.

6.2 Check-in sem app (cadastro rápido)



Campos do cadastro rápido — objetivo, cabe em uma tela:

1. Nome da criança · data de nascimento (ou idade) · sexo
2. Comorbidade / alergia · medicação em uso · deficiência *(um campo de texto cada, todos opcionais)*
3. Responsável: nome · telefone (com "tem WhatsApp?" marcado por padrão) · parentesco
4. Contato de emergência alternativo (opcional)
5. Quem mais pode retirar (texto livre, opcional)

6.3 Check-out



Syntax error in text
mermaid version 11.17.2

6.4 Anotações

- **Individual:** vinculada ao `checkinId` (e portanto à sessão e à criança). Chave `visibleToGuardian` decide se aparece para o responsável.
- **Geral da aula:** `kind = CLASS`, sem `childId`. Vai para todos os responsáveis das crianças presentes naquela sessão, ao fechar a sala.

6.5 Alerta escalonado

Nível	Quando usar	Canais disparados	Confirmação
1 • INFO	"Trouxe lanche?", "Precisa de fralda"	Push (se tem app) → WhatsApp se sem app ou sem leitura em 3 min	Opcional
2 • URGENT	"Está chorando muito", "Passou mal"	Push + WhatsApp em paralelo, sempre os dois	Pedida; se não houver em 2 min, tela do professor sugere ligar
3 • EMERGENCY	Acidente, convulsão, criança sumida	Push crítico + WhatsApp + discagem automática na tela do professor + alerta na tela de todos os ADMIN logados	Obrigatória; alerta fica OPEN até alguém resolver

Toda tentativa vira uma linha em **KidsAlertDelivery** — é o rastro de que a igreja tentou avisar, o que importa juridicamente se algo der errado.

7. API REST

Prefixo `/v1/kids`, atrás de `authMiddleware + requireFeature('kids')`. Padrão de erro e envelope seguem o restante da API (`AppError, { error: { code, message } }`).

7.1 Salas

Método	Rota	Papel	Descrição
GET	<code>/rooms</code>	KIDS, ADMIN	Lista salas com ocupação da sessão aberta
POST	<code>/rooms</code>	ADMIN	Cria sala
PATCH	<code>/rooms/:id</code>	ADMIN	Edita sala
DELETE	<code>/rooms/:id</code>	ADMIN	Desativa (soft delete via <code>isActive</code>)
PUT	<code>/rooms/:id/teachers</code>	ADMIN	Substitui a lista de professores

7.2 Sessões

Método	Rota	Papel	Descrição
POST	<code>/rooms/:id/sessions</code>	KIDS, ADMIN	Abre a aula (<code>serviceDate</code> , <code>serviceName</code> , <code>lesson</code>)
GET	<code>/sessions/:id</code>	KIDS, ADMIN	Sessão + lista de check-ins + notas
PATCH	<code>/sessions/:id</code>	KIDS, ADMIN	Edita lição/nome do culto

Método	Rota	Papel	Descrição
POST	/sessions/:id/close	KIDS, ADMIN	Fecha; 409 se houver CHECKED_IN
GET	/sessions	ADMIN	Histórico com filtros (sala, período)

7.3 Crianças e responsáveis

Método	Rota	Papel	Descrição
GET	/children/search?q=	KIDS, ADMIN	Busca por nome da criança ou telefone do responsável
POST	/children/quick	KIDS, ADMIN	Cadastro rápido (§6.2)
GET	/children/:id	KIDS (em sala), ADMIN, RESPONSÁVEL (filho)	Ficha completa
PATCH	/children/:id	ADMIN, RESPONSÁVEL (filho)	Edita ficha/saúde
POST	/children/:id/guardians	ADMIN, RESPONSÁVEL	Adiciona responsável
POST	/children/:id/link-user	ADMIN	Vincula criança do balcão a uma conta criada depois

7.4 Check-in / check-out

Método	Rota	Papel	Descrição
POST	/checkins/resolve-qr	KIDS	Valida QR e devolve filhos elegíveis
POST	/checkins	KIDS	Confirma check-in de 1..N crianças
POST	/checkins/:id/checkout	KIDS	QR, senha ou liberação manual
GET	/checkins/:id	KIDS, ADMIN	Detalhe + anotações + alertas
POST	/checkins/:id/regenerate-code	KIDS, ADMIN	Nova senha (pai perdeu o papel); invalida a anterior

POST /v1/kids/checkins — corpo e resposta:

```
// request
{
  "sessionId": "uuid",
  "childIds": ["uuid", "uuid"],
  "guardianId": "uuid",           // quem entregou
  "method": "QR",                 // QR | CODE | MANUAL
  "qrToken": "eyJhbGciOiIi...    // obrigatório quando method = QR
}
```

```
// 201
{
  "checkins": [
    {
      "id": "uuid",
      "childId": "uuid",
      "childName": "Ana Beatriz",
      "badgeCode": "K-014",
      "pickupCode": "48213",      // ← só aqui, uma única vez; depois só o
hash
      "healthFlags": ["alergia: amendoim", "uso contínuo: ritalina"]
    }
  ],
  "roomOccupancy": { "current": 14, "capacity": 20 }
}
```

Erros específicos: 409 KIDS_ROOM_FULL, 409 KIDS_ALREADY_CHECKED_IN, 422 KIDS_AGE_OUT_OF_RANGE (com `force: true` para o professor sobrepor), 401 KIDS_QR_EXPIRED, 409 KIDS_QR_REPLAY.

7.5 Anotações

Método	Rota	Papel
POST	/sessions/:id/notes	KIDS
PATCH	/notes/:id	KIDS (autor), ADMIN
DELETE	/notes/:id	ADMIN
GET	/children/:id/notes	RESPONSÁVEL (filho), KIDS, ADMIN

7.6 Alertas

Método	Rota	Papel	Descrição
POST	/checkins/:id/alerts	KIDS	Cria alerta e enfileira entregas
POST	/alerts/:id/acknowledge	RESPONSÁVEL	"Estou indo"
POST	/alerts/:id/resolve	KIDS, ADMIN	Encerra
GET	/alerts?status=OPEN	KIDS, ADMIN	Painel de alertas abertos
GET	/alerts/:id/deliveries	ADMIN	Rastro de entrega por canal

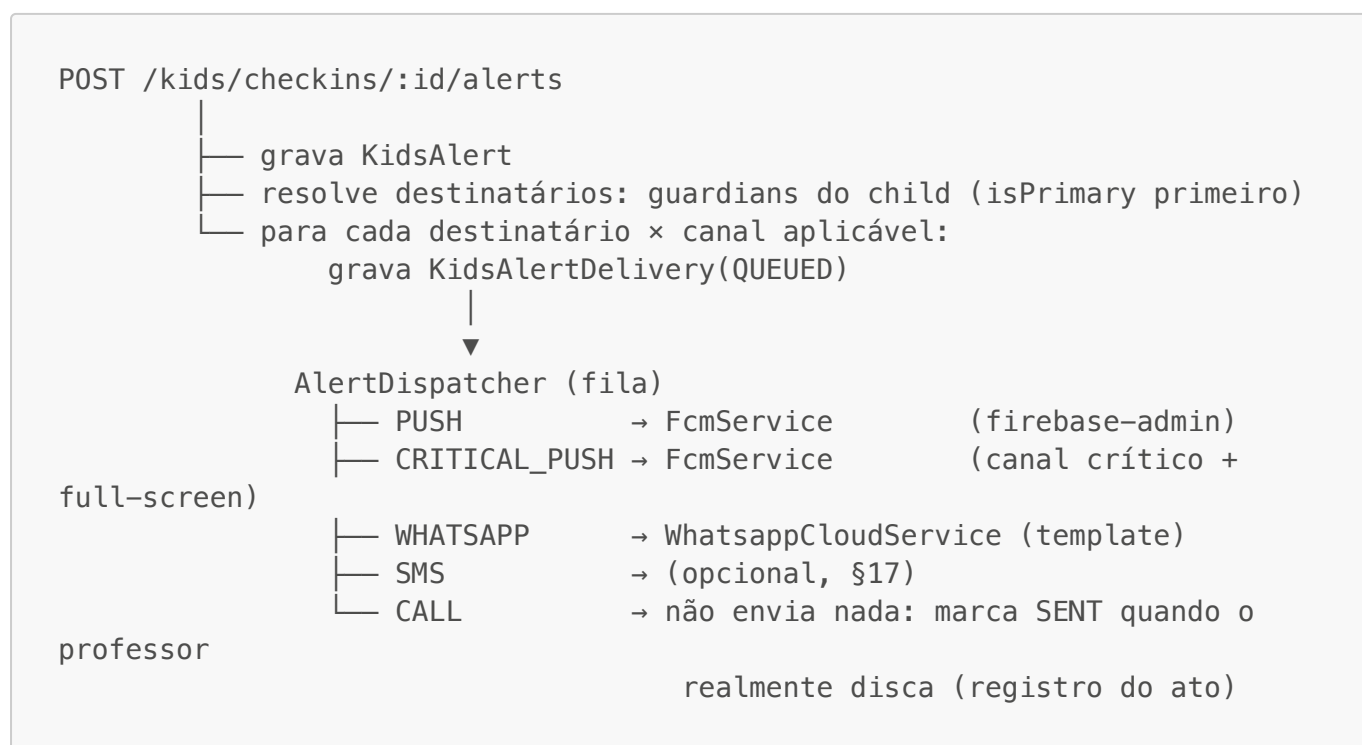
7.7 Dispositivos e QR do responsável

Método	Rota	Papel	Descrição
POST	/v1/devices	qualquer autenticado	Registra/atualiza token FCM

Método	Rota	Papel	Descrição
DELETE	/v1/devices/:token	qualquer autenticado	Logout/desinstalação
GET	/v1/kids/my-qr	RESPONSÁVEL	Token curto para o QR (TTL 60 s)
GET	/v1/kids/my-children	RESPONSÁVEL	Filhos + status atual (em qual sala está agora)

8. Alertas e notificações

8.1 Arquitetura de entrega



Fila: v1 pode rodar in-process com retry exponencial (3 tentativas: 0 s, 30 s, 2 min), porque o volume é baixo (dezenas por culto). Se virar gargalo, trocar por BullMQ/Redis sem mudar o contrato — o **AlertDispatcher** é uma interface no domínio.

8.2 Push (FCM)

Novo **IPushService** no domínio; implementação **FcmPushService** com **firebase-admin**. Payload:

```

{
  "notification": { "title": "Kids 4-6 · Ana Beatriz", "body": "Está com febre" },
  "data": {
    "type": "kids_alert",
    "level": "URGENT",
    "alertId": "uuid",
    "childId": "uuid",
    "deepLink": "/kids/alerts/uuid"
  }
}
  
```

```

},
"android": {
  "priority": "high",
  "notification": {
    "channelId": "kids_urgent",           // ou kids_emergency
    "sound": "alerta",
    "visibility": "public"
  }
},
"apns": {
  "headers": { "apns-priority": "10", "apns-push-type": "alert" },
  "payload": { "aps": {
    "sound": { "critical": 1, "name": "alerta.caf", "volume": 1.0 }, //
só com entitlement
    "interruption-level": "critical"                                //
fallback: time-sensitive
  } }
}
}

```

Canais Android (criados no primeiro boot do app):

Canal	Importância	Uso
<code>kids_info</code>	DEFAULT	Nível 1
<code>kids_urgent</code>	HIGH	Nível 2
<code>kids_emergency</code>	HIGH + bypassDnd + fullScreenIntent + som contínuo	Nível 3

8.3 WhatsApp (Cloud API)

Alerta chega **fora da janela de 24 h**, então só passa com **template pré-aprovado** (categoria *Utility*).
Três templates a submeter:

Nome	Corpo	Variáveis
<code>kids_checkin_confirmado</code>	"Olá {{1}}! {{2}} foi recebido(a) na sala {{3}} às {{4}}. Senha de retirada: {{5}}. Guarde esta mensagem."	5
<code>kids_alerta_urgente</code>	"Aviso da sala {{1}} sobre {{2}}: {{3}}. Por favor, dirija-se à salinha."	3
<code>kids_emergencia</code>	"EMERGÊNCIA na sala {{1}} sobre {{2}}: {{3}}. Estamos ligando para você agora."	3

Config nova em `.env`: `WHATSAPP_PROVIDER`, `WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_TEMPLATE_NAMESPACE`. O envio manual atual (`wa.me`) continua existindo para campanhas — os dois convivem.

Custo: conversa de utilidade é cobrada por mensagem iniciada pelo negócio. Estimar por igreja: `no crianças sem app × alertas por culto × cultos/mês`. Entra na conta do plano (§12).

8.4 Ligação

Nível 3 abre no app do professor um botão "Ligar agora" (**tel:** via **url_launcher**, já no projeto) com o telefone do responsável primário. Ao confirmar a discagem, o app faz **POST** **/alerts/:id/deliveries/call** registrando o ato. Nada de discagem silenciosa: a Play Store rejeita e, mais importante, alguém precisa falar.

9. Segurança

9.1 QR do responsável

- **Formato:** JWT compacto assinado (HS256) com **sub** = **userId**, **chr** = **churchId**, **jti** = **uuid**, **exp** = 60 s. Gerado pela API (**GET /kids/my-qr**) — não pelo app, para a chave não sair do servidor.
- **Renovação:** a tela regenera a cada 45 s enquanto aberta.
- **Antirreplay:** **jti** consumido grava em **kids_qr_nonces**; segunda leitura → **409 KIDS_QR_REPLAY**. Limpeza por **expiresAt**.
- **Print/foto do QR:** inútil após 60 s. Sem TTL curto, um print vira chave permanente da criança.
- **Tenant:** **chr** do token tem de bater com o **churchId** do professor.

9.2 Senha de 5 dígitos

5 dígitos = 100 000 combinações — fraco se atacado à força bruta, e é **por decisão de produto** (precisa ser digitável e ditável). Controles obrigatórios:

- Hash (**bcrypt**, custo 10) — nunca em texto puro no banco nem em log.
- Exibida **uma única vez** na resposta do check-in; depois só **pickupCodeLast2** para conferência visual.
- Escopo mínimo: vale **para aquele check-in**, naquela sessão, naquela sala.
- Rate limit: 5 tentativas → bloqueio de 15 min (**pickupLockedUntil**); alerta ao ADMIN no 3º erro.
- Regeneração invalida a anterior.
- Retirada sempre exige **nome de quem retirou** registrado no check-out.

9.3 Autorização por sala

Todo endpoint **KIDS** passa por um guard:

```
// api/src/infrastructure/http/middlewares/kids.middleware.ts
requireRoomAccess(sessionOrRoomId) // ADMIN passa; KIDS só se estiver em
kids_room_teachers
```

9.4 Auditoria

Check-in, check-out, liberação manual, alerta e edição de dado de saúde geram registro imutável (append-only). v1: **KidsCheckin/KidsAlert** já guardam quem e quando; para edição de ficha, gravar **KidsAuditLog** (§17) se o cliente exigir.

10. LGPD e dados de saúde

Alergia, medicação e deficiência são **dado pessoal sensível** (LGPD art. 5º, II) de **criança** (art. 14 — melhor interesse do menor). Isso impõe:

Exigência	Como atendemos
Base legal	Consentimento específico do responsável, coletado no cadastro rápido e no app, com texto próprio — não no meio do termo geral
Finalidade	Só cuidado durante a aula. Proibido usar em relatório, campanha ou export
Minimização	Campos de saúde são opcionais; a tela diz "informe apenas o necessário para o cuidado"
Acesso	Visível só para o professor da sala onde a criança está agora e ADMIN; sai da tela ao fazer check-out
Retenção	Ficha ativa enquanto a criança frequenta; medicalNotes some após 12 meses sem check-in (job)
Foto	Opcional e desligada por padrão na configuração da igreja; se ligada, exige consentimento à parte
Titular	Responsável pode ver, corrigir e pedir exclusão pelo app (PATCH /children/:id , DELETE com anonimização)
Vazamento	Registro de acesso a ficha com dado sensível (quem abriu, quando)

Nada de dado de saúde em push ou WhatsApp. O alerta diz "passou mal, venha à sala"; o detalhe fica no app, atrás de autenticação. Mensagem em tela de bloqueio é lida por qualquer um que pegue o celular.

11. App Flutter

11.1 Dependências novas

Pacote	Uso	Observação
mobile_scanner	Ler QR no app do professor	Não existe no projeto; qr_flutter (já presente) só gera
firebase_messaging	Push	Já no pubspec , sem uso — precisa inicialização
flutter_local_notifications	Canal crítico, full-screen intent	Já no pubspec
permission_handler	Câmera, notificação, política de DND	Novo

11.2 Estrutura (clean arch, igual às outras features)

```

lib/features/kids/
├── data/
│   ├── models/                kids_room_model, child_model, checkin_model,
alert_model
│   └── repositories/         kids_repository_impl.dart
├── domain/
│   ├── entities/            kids_room, kids_session, kids_child,
kids_checkin, kids_alert
│   └── repositories/        i_kids_repository.dart
└── presentation/
    ├── pages/
    │   ├── kids_home_page.dart        // professor: minhas salas +
sessão aberta
    │   ├── kids_session_page.dart     // lista de crianças na sala,
ocupação, ações
    │   ├── kids_scan_page.dart        // câmera (check-in e check-out)
    │   ├── kids_quick_register_page.dart // cadastro rápido
    │   └── kids_child_detail_page.dart // ficha + saúde + anotações +
alerta
    │   ├── kids_rooms_admin_page.dart // ADMIN: salas e professores
    │   └── parent_children_page.dart  // responsável: meus filhos +
status
    │   ├── parent_qr_page.dart        // QR rotativo
    │   └── parent_alerts_page.dart    // alertas recebidos + "estou
indo"
    └── widgets/
        ├── kids_occupancy_bar.dart
        ├── kids_child_tile.dart
        ├── kids_pickup_code_dialog.dart
        ├── kids_alert_sheet.dart      // escolher nível + mensagem
        └── kids_health_badges.dart

```

11.3 Rotas (`app_router.dart`)

<code>/kids</code>	→ <code>KidsHomePage</code>	(KIDS, ADMIN)
<code>/kids/sessions/:id</code>	→ <code>KidsSessionPage</code>	(KIDS, ADMIN)
<code>/kids/scan</code>	→ <code>KidsScanPage</code>	(KIDS)
<code>/kids/children/:id</code>	→ <code>KidsChildDetailPage</code>	(KIDS, ADMIN)
<code>/admin/kids/rooms</code>	→ <code>KidsRoomsAdminPage</code>	(ADMIN)
<code>/meus-filhos</code>	→ <code>ParentChildrenPage</code>	(RESPONSAVEL)
<code>/meus-filhos/qr</code>	→ <code>ParentQrPage</code>	(RESPONSAVEL)
<code>/meus-filhos/alertas</code>	→ <code>ParentAlertsPage</code>	(RESPONSAVEL)

O `redirect` do router ganha o papel `RESPONSAVEL`, cuja home é `/meus-filhos` — essa conta **não** vê dashboard, células nem visitantes.

11.4 Comportamento offline

A salinha costuma ter Wi-Fi ruim. v1 assume online, mas com dois amortecedores:

- **Check-in otimista:** a UI marca a criança e a requisição vai numa fila local; falha exibe banner "3 check-ins pendentes" com retry manual.
- **Ficha em cache:** dados da sessão aberta ficam em memória para a lista não sumir a cada reconexão.

Sincronização offline completa (com resolução de conflito) fica para depois — é onde mora a complexidade real e não vale para o v1.

11.5 Design

Reusar o design system existente (**AppCard**, **AppBadge**, **StatCard**, **AppSectionHeader**). Regras próprias do módulo:

- Ocupação: barra + **14/20**; passa a âmbar em 90 %, vermelho ao lotar.
- Marcadores de saúde: badge vermelha para alergia, âmbar para medicação, azul para deficiência — sempre com o texto, nunca só a cor.
- Botão de alerta: sempre visível na ficha, com os 3 níveis nomeados e uma confirmação extra no nível 3 (segurar 2 s para disparar).

12. Plano SaaS e feature gating

Nova chave em **api/src/shared/plans/features.ts**:

```
export const FEATURES = {  
  // ...existentes  
  KIDS: 'kids',  
} as const;  
  
// FEATURE_CATALOG  
{  
  key: FEATURES.KIDS,  
  label: 'Kids (salinha infantil)',  
  description: 'Check-in/check-out de crianças, anotações e alertas aos responsáveis.',  
}
```

Sugestão de disponibilidade e limites (o dono decide):

Tier	Kids	Salas	Alertas WhatsApp/mês	Push
FREE	—	—	—	—
STARTER	—	—	—	—
GROWTH	✓	3	200	ilimitado
COMPLETE	✓	ilimitado	1 000	ilimitado

O limite de salas é verificado em **POST /kids/rooms**; o de WhatsApp, no **AlertDispatcher** (excedeu → cai para push apenas e avisa o ADMIN). No app,

`ChurchContextController.hasFeature('kids')` esconde o menu.

13. Impacto no código existente

Arquivo	Mudança
<code>api/prisma/schema.prisma</code>	10 modelos novos, 9 enums, <code>Child</code> estendido, <code>UserRole</code> + 2 valores
<code>api/src/infrastructure/database/tenant-guard.ts</code>	Adicionar aos <code>TENANT_MODELS</code> : <code>KidsRoom</code> , <code>KidsRoomTeacher</code> , <code>KidsSession</code> , <code>KidsGuardian</code> , <code>KidsCheckin</code> , <code>KidsNote</code> , <code>KidsAlert</code> , <code>KidsAlertDelivery</code> , <code>DeviceToken</code>
<code>api/src/infrastructure/http/app.ts</code>	<code>app.use(\{v1}/kids\`, kidsRoutes(...))` e <code>`app.use(\{v1}/devices\`, deviceRoutes(...))`</code></code>
<code>api/src/shared/container/index.ts</code>	Registrar repositórios, use cases, <code>FcmPushService</code> , <code>WhatsappCloudService</code> , <code>AlertDispatcher</code>
<code>api/src/shared/plans/features.ts</code>	Chave <code>kids</code> + catálogo
<code>api/src/infrastructure/http/middlewares/kids.middleware.ts</code>	Novo: <code>requireRoomAccess</code>
<code>api/src/infrastructure/http/middlewares/auth.middleware.ts</code>	<code>requireKids</code> , e revisar <code>requireStaff</code> para não incluir <code>RESPONSABLE</code>
<code>lib/features/auth/domain/entities/user_entity.dart</code>	<code>UserRole.kids</code> , <code>UserRole.responsavel</code> + <code>fromString</code>
<code>lib/routing/app_router.dart</code>	Rotas §11.3 + redirect por papel
<code>lib/injection/injection.dart</code>	Repositório Kids, serviço de push

Arquivo	Mudança
<code>lib/main.dart</code>	Init do Firebase + handler de mensagem em background
<code>pubspec.yaml</code>	<code>mobile_scanner</code> , <code>permission_handler</code>
<code>docs/screen-map.md</code> / <code>.mmd</code>	Novas telas no mapa
<code>api/prisma/seed*.ts</code>	Sala de exemplo + usuário KIDS para ambiente de dev

Ponto de atenção: `authMiddleware/requireStaff` hoje tratam qualquer autenticado como membro da equipe. Com `RESPONSAVEL` isso deixa de valer — cada rota existente precisa ser revista para não vaziar dado de célula para pai. É a mudança mais arriscada do módulo em código já em produção.

14. Fases de entrega

Fase	Entrega	Depende de	Estimativa*
F0 · Infra	FCM ponta a ponta (API + app + <code>device_tokens</code>), WhatsApp Cloud API com 1 template, <code>mobile_scanner</code>	Conta Meta Business, projeto Firebase	1,5 sem
F1 · Cadastro	Salas, professores, <code>Child</code> estendido, papéis <code>KIDS/RESPONSAVEL</code> , telas de admin	F0 parcial	1,5 sem
F2 · Check-in/out	QR rotativo, cadastro rápido, senha de 5 dígitos, sessões, ocupação	F1	2 sem
F3 · Anotações	Individual e geral, visibilidade ao responsável, app do pai	F2	0,5 sem
F4 · Alertas 1–2	<code>KidsAlert</code> , dispatcher, push + WhatsApp, confirmação	F0, F2	1,5 sem
F5 · Emergência	Canal crítico Android, ligação registrada, painel de alertas abertos, pedido de entitlement iOS	F4	1 sem
F6 · Relatórios	Frequência infantil, histórico por criança, export	F2	1 sem

* Estimativa de um desenvolvedor, sem QA dedicado. F0 pode atrasar por aprovação de template do WhatsApp (dias) e do entitlement da Apple (semanas, incerto).

Ordem de valor: F1+F2 já entregam o controle de entrega — que é o coração do pedido. F4 é o que exige a infra mais cara.

15. Testes e critérios de aceite

15.1 Casos que precisam de teste automatizado (API)

- QR expirado → 401; QR reusado → 409; QR de outra igreja → 403.
- Senha correta → check-out; 5 erradas → bloqueio; senha de outro check-in → 401.
- Sala cheia → 409, com **force** só para ADMIN.
- Mesma criança em duas sessões abertas → 409.
- Fechar sessão com criança dentro → 409.
- Professor fora de **kids_room_teachers** → 403 em toda a rota da sala.
- Tenant: professor da igreja A não enxerga sessão da igreja B (o **tenant-guard** cobre, mas o teste precisa existir — é dado de criança).
- Alerta gera N **KidsAlertDelivery** conforme canais elegíveis do responsável.
- Falha do provedor WhatsApp → **FAILED** com erro, sem derrubar a requisição.

15.2 Critérios de aceite (produto)

1. Professor faz check-in de uma criança com QR do pai em **menos de 10 s**.
2. Cadastro rápido de criança nova em **menos de 60 s**, com senha exibida ao fim.
3. Senha de 5 dígitos digitada errada 5x bloqueia e avisa o ADMIN.
4. Alerta nível 2 chega no celular do pai com app em **até 5 s**; sem app, o WhatsApp sai em **até 30 s**.
5. Nível 3 abre discagem no aparelho do professor com um toque e registra a ligação.
6. Nenhuma tela mostra dado de saúde para quem não é professor da sala/ADMIN/pai.
7. Fechar a sala lista quem ainda está dentro e impede o fechamento.
8. Relatório do culto mostra: entradas, saídas, alertas e pendências.

16. Observabilidade e operação

- **Logs estruturados** (padrão atual do **request-logger**) com **sessionId**, **checkinId**, **alertId** — **nunca** com senha, QR ou dado de saúde.
- **Métricas por culto**: crianças por sala, tempo médio de check-in, alertas por nível, taxa de entrega por canal, retiradas manuais (indicador de problema).
- **Alarme operacional**: falha de entrega > 20 % em uma sessão → e-mail ao ADMIN.
- **Job noturno**: limpar **kids_qr_nonces** expirados; anonimizar **medicalNotes** de crianças inativas há 12 meses.
- **Backup**: as tabelas do Kids entram no mesmo dump do Postgres; nada de export de dado de saúde para planilha.

17. Riscos e decisões pendentes

17.1 Riscos

Risco	Impacto	Mitigação
Entitlement de Critical Alerts negado pela Apple	Nível 3 no iPhone fica como time-sensitive	Ligação telefônica é o canal primário do nível 3

Risco	Impacto	Mitigação
Template de WhatsApp reprovado	Alerta sem app não sai	Submeter cedo (F0); fallback SMS
Wi-Fi ruim na salinha	Check-in trava	Fila otimista (§11.4); tablet com 4G
Senha de 5 dígitos fotografada/compartilhada	Retirada indevida	Escopo por sessão, nome de quem retirou, incentivo ao app
RESPONSÁVEL enxergando dado de célula	Vazamento	Revisão de requireStaff em todas as rotas (§13)
Custo de WhatsApp acima do previsto	Margem do plano	Limite por tier + corte automático para push

17.2 Decisões pendentes (precisam do dono)

1. **Papel KIDS_COORD** (coordenador do ministério, vê todas as salas) entra no v1 ou o **ADMIN** cobre?
2. **Foto da criança** no check-in: liga ou não? Aumenta a segurança da entrega e o risco de LGPD.
3. **Etiqueta impressa** (pulseira/crachá com **badgeCode**): imprimir por Bluetooth ou só mostrar na tela?
4. **Multi-filho**: um QR entrega todos os filhos de uma vez ou um por vez? (Proposta: tela mostra todos, professor marca quem fica.)
5. **Tier mínimo** do Kids: GROWTH (proposto) ou COMPLETE?
6. **SMS como fallback** quando o responsável não tem WhatsApp: entra? Tem custo e exige provedor novo.
7. **Auto-cadastro do responsável**: o pai cria a própria conta (como o **signup** público de igreja) ou o ADMIN cadastra e envia convite?

Referências no repositório

- Multi-tenant e planos: **SAAS_PLANO.md**
- Envio de WhatsApp atual (manual, **wa.me**): **WHATSAPP_ENVIO_LOTE.md**
- Isolamento por igreja: **api/src/infrastructure/database/tenant-guard.ts**
- Gating por plano: **api/src/infrastructure/http/middlewares/feature.middleware.ts**
- Mapa de telas: **screen-map.md**